NANDI

Reliable. Shared. Safe.

Project Audit Report

Full-Stack Security · Bug Analysis · Performance · Roadmap

TOTAL ISSUES	CRITICAL	HIGH	MEDIUM	LOW
57	5	22	18	12

Generated	April 23, 2026 21:49
App	NANDI Rideshare (Expo SDK 54 + tRPC + Drizzle/MySQL)
Scope	Full-stack: Server, Client (React Native/Web), DB schema, Infra
Auditor	Claude Sonnet 4.6 — Automated Static Analysis

Top Priority Actions

1. CRITICAL: Rotate exposed Stripe keys in .env immediately — add .env to .gitignore
2. CRITICAL: Fix memory leak in dispatch queue — pendingForDriver orphaned entries
3. CRITICAL: Fix race condition in joinRide — use atomic seat decrement SQL
4. CRITICAL: Swallowed errors in fare update — add logging and retry queue
5. HIGH: Include sessionVersion in JWT to invalidate tokens on password change
6. HIGH: Add Stripe payment amount validation in webhook handlers

© 2026 NANDI · Internal Use Only

TABLE OF CONTENTS

1	**Executive Summary**	Security overview and risk posture
2	**Security Findings**	Auth, API, SQL, secrets, Stripe
3	**Critical Bugs & Runtime Errors**	Crashes, data loss, race conditions
4	**TypeScript & Type Safety**	Type gaps, unsafe casts, unhandled promises
5	**Performance Issues**	N+1 queries, polling, bundle size
6	**Incomplete Implementations**	TODOs, stubs, unfinished flows
7	**Infrastructure & Config**	Redis, env vars, Google Maps, push
8	**Advanced Feature Roadmap**	Prioritised suggestions for v2+
9	**Appendix**	File manifest, tech stack

© 2026 NANDI · Internal Use Only

2 SECURITY FINDINGS

Authentication · API Guards · SQL · Secrets · Stripe

Hardcoded Stripe Keys Committed in .env

`.env:10-13`

Real Stripe secret key (sk_test_51TFbnt...) and publishable key are in the .env file. If ever committed to git or leaked, all payment data is compromised. The webhook secret is a placeholder but still concerning.

Fix: Move all Stripe keys to a secrets manager (AWS Secrets Manager, 1Password). Rotate the exposed keys immediately. Never commit credentials — add .env to .gitignore and verify with `git rm --cached .env`.

Weak JWT Secret in Development

`.env:3`

JWT_SECRET is 'supersecret_dev_key_change_me' — a default insecure value. If this ever reaches production, session hijacking is trivial. JWT tokens signed with this key are cryptographically weak.

Fix: Enforce in server/_core/env.ts that JWT_SECRET must be >= 32 characters and is not the default placeholder. Already partially done — add regex check and fail fast on startup.

Missing Payment Amount Validation in Stripe Webhook

`server/services/stripe.ts:559-573`

handlePaymentIntentSucceeded() trusts the payment intent status without verifying the amount matches what was requested. An attacker who can craft a webhook event claiming a partial payment succeeded would receive the full service.

Fix: Compare pi.amount against the expected amountCents stored in stripePayments record before updating status to 'succeeded'. Reject if delta > 1 cent.

No Rate Limiting on OTP Endpoints

`server/_core/index.ts:114-163`

OTP requests (auth.requestOtp) share the global 2000 req/60s IP-based rate limit. An attacker can enumerate and brute-force 6-digit OTPs at scale before the limit triggers.

Fix: Add per-endpoint, per-identifier rate limiting on OTP: max 3 requests per 15 minutes per phone/email. Use Redis-backed sliding window with exponential backoff after each failure.

sessionVersion Excluded from JWT Payload — Password Changes Don't Invalidate Tokens

`server/routers.ts:513`

TODO comment confirms sessionVersion is NOT included in JWT payload. A user who changes their password or logs out on one device keeps all other active sessions live indefinitely until TTL expires.

Fix: Include sessionVersion in JWT payload at issuance. Validate it on every protectedProcedure. Increment sessionVersion in DB on password change or explicit logout.

Unvalidated Multipart Parser — Custom Implementation

`server/_core/index.ts:278-338`

Custom multipart body parser uses raw regex on HTTP headers without boundary length checks. Malformed Content-Type headers or crafted boundaries could cause incorrect parsing or skip file data silently.

Fix: Replace with busboy or formidable — battle-tested libraries. Add file size limit (5MB), MIME type whitelist, and filename sanitization.

© 2026 NANDI · Internal Use Only

localStorage Used for Session Tokens on Web

`lib/_core/auth.ts:84,108-109`

On web platform, session tokens are stored in localStorage which is accessible to any XSS attack. No HttpOnly equivalent exists in browser storage.

Fix: Switch to httpOnly, secure, SameSite=Strict cookies for web sessions. Keep in-memory cache for performance but never write tokens to localStorage.

Stripe Webhook Secret Placeholder Not Rejected at Startup

`server/services/stripe.ts:659-661`

Server warns if STRIPE_WEBHOOK_SECRET is missing but doesn't reject the placeholder string 'whsec_YOUR_WEBHOOK_SECRET_HERE'. Webhooks pass unchecked in dev with wrong secret.

Fix: In env.ts, throw at startup if STRIPE_WEBHOOK_SECRET is unset or contains placeholder substring. Fail fast.

No CSRF Protection on State-Changing Mutations

`server/_core/index.ts:1-100`

No CSRF tokens or SameSite cookie enforcement documented. CORS allows credentials. State-changing tRPC mutations are vulnerable to CSRF from malicious third-party sites.

Fix: Add X-CSRF-Token header validation on all mutations. Set SameSite=Strict on session cookies. Validate Origin header against allowlist.

Type Casts Bypass Stripe SDK Discriminated Unions

`server/services/stripe.ts:468,485-486`

Multiple 'as any' casts when handling Stripe webhook events hide type mismatches. 'event.data.object as any' skips the SDK's event type validation entirely.

Fix: Narrow event type using discriminated union: switch(event.type) { case 'payment_intent.succeeded': const pi = event.data.object as Stripe.PaymentIntent; ... }

© 2026 NANDI · Internal Use Only

3 CRITICAL BUGS & RUNTIME ERRORS

Crashes · Data Loss · Race Conditions · Null Dereferences

Memory Leak: pendingForDriver Map Never Cleaned Up on Ride Completion

`server/services/driverMatcher.ts:53-55,129,162,178,190,205`

driverId → rideId mapping is added at line 129 but only deleted on accept (162), decline (178), timeout (190), or cancel (205). If a ride completes naturally (driver side) with a pending dispatch entry still open, the entry leaks forever.

Fix: Add cancelDispatch() call in completeRide and driver.completeRide procedures. Add a scheduled sweep every 60s to remove entries where rideId is in terminal state.

Race Condition: Concurrent joinRide Can Exceed Seat Limit

`server/routers.ts:1512-1520`

Two simultaneous join requests both see existing.length < 3 and both succeed, adding a 4th passenger. The existing.length check is a read followed by a non-atomic write.

Fix: Replace read-then-write with atomic SQL: UPDATE rides SET availableSeats = availableSeats - 1 WHERE id = ? AND availableSeats > 0. Check affectedRows before inserting passenger record.

Silent Fare Update Failure: Swallowed Errors in Mid-Ride Join

`server/routers.ts:2274-2281`

updateRidePassengerFare() calls wrapped in .catch(() => {}) silently swallow all errors. If fare updates fail for mid-ride joins, passengers continue with stale fares with no logging, no retry, and no alert.

Fix: Log errors at warn level with rideId and passengerId context. Implement retry with exponential backoff (max 3 attempts). Surface failed updates to admin alert.

No Null Check on Ride Object After OSRM Lookup

`server/routers.ts:1352`

After bookRide creates a ride, OSRM lookup (lines 1347-1358) can fail silently. ride.id is used without verifying ride was actually created and has expected structure.

Fix: Add explicit null check: if (!ride?.id) throw new TRPCError({code: 'INTERNAL_SERVER_ERROR'}). Validate ride object shape matches expected type before proceeding.

Division by Zero Risk in Shared Ride Fare Splitting

`server/routers.ts:1535-1536`

updatedPassengers.length is used as divisor after adding a new passenger. If getRidePassengers() returns empty list (DB error), fare calculation produces Infinity/NaN which propagates to payment.

Fix: Guard: if (updatedPassengers.length === 0) throw new TRPCError({code: 'INTERNAL_SERVER_ERROR', message: 'Passenger count error'}). Validate count > 0 before division.

Out-of-Bounds Access in Hourly Earnings Heatmap

`server/routers.ts:3492-3493`

buckets[hour]! uses non-null assertion without validating hour is in [0-23]. Invalid timestamp (e.g., TZ conversion edge case) could produce hour = NaN or 24, causing undefined[].total += crash.

Fix: Clamp: const hour = Math.max(0, Math.min(23, new Date(r.completedAt).getHours())); Remove non-null assertion and add optional chaining.

© 2026 NANDI · Internal Use Only

Driver Name Not Sanitized Before WebSocket Broadcast

`server/services/websocket.ts:181`

senderName from socket payload is sent directly in notification message without sanitization. Specially crafted names with JSON-breaking characters could corrupt SSE parsing client-side.

Fix: Sanitize: senderName = (senderName as string).replace(/[<>"'&]/g, '').slice(0, 100). JSON.stringify the notification payload before sending.

cancelRide Allows Cancelling Already Completed Rides

`server/routers.ts:1571-1596`

cancelRide() calls atomicTransitionRideStep which should catch terminal states, but the passenger-facing code path doesn't check for 'completed' status before attempting cancel. Edge case can still trigger passenger notifications for completed rides.

Fix: Add explicit early exit: if (isTerminalStep(ride.rideStep)) throw new TRPCError({code: 'BAD_REQUEST', message: 'Ride already ' + ride.rideStep}).

Recurring Ride Failures Silently Skipped — No Retry Queue

`server/_core/index.ts:576-578`

Recurring ride creation catches exceptions but only logs a warning. Failed recurring rides are permanently lost with no retry, dead-letter queue, or admin alert.

Fix: Add failed recurring rides to a retry_queue table. Implement exponential backoff (5m, 30m, 2h). Alert admin after 3 consecutive failures on same recurring ride.

estimateFare() Uses Oversimplified Duration Estimate

`server/routers.ts:312-320`

Duration estimated as distanceKm * 2 + 5 ignoring traffic, speed limits, and route complexity. Used for both fare calculation and driver ETA display — inaccurate in urban areas.

Fix: Pass actual durationMinutes from OSRM route response (already fetched in bookRide). Fall back to formula only when OSRM unavailable.

© 2026 NANDI · Internal Use Only

4 TYPESCRIPT & TYPE SAFETY

Unsafe Casts · Unhandled Promises · Missing Types

40+ 'as any' Casts Hide Real Type Errors Throughout routers.ts

`server/routers.ts:471,1343,1352,1596,1603,1697,1950,1976,2047-2048`

Extensive use of 'as any' bypasses TypeScript's type safety. Examples: ctx cast, ride object properties assumed, join request properties, globalThis access. These hide real bugs that TS would otherwise catch at compile time.

Drizzle Schema Stores Numeric Fields as VARCHAR — Implicit Type Mismatch

`drizzle/schema.ts (rides table)`

pickupLat, pickupLng, dropoffLat, dropoffLng, distanceKm are declared as varchar() in schema but treated as numbers in application code via Number() casts. A corrupt DB value ('NaN', empty string) will silently return 0 from Number().

Unhandled Promise Rejections in rideMonitor Async Callbacks

`server/services/rideMonitor.ts:69-73`

onGeofenceArrival, onNoShowCancel, onRideSearchTimeout are async callbacks called without await or .catch(). Unhandled rejections in Node.js 20 crash the process.

Missing Return Type Annotations on Exported Helper Functions

`server/routers.ts:312-321 (estimateFare), 322-332 (haversineKm)`

Critical helper functions lack explicit return type annotations. TypeScript's inferred types can drift when the function body changes, causing silent type mismatches downstream.

globalThis Pollution Used as Ad-Hoc State Store

`server/routers.ts:1697-1698,2912-2924,2995-2998`

globalThis.__locationThrottle, __etaThrottle, __deviationThrottle used as type 'any'. No type declaration, no cleanup on shutdown, and no collision protection with other modules.

tRPC Context User Type Is Nullable But Code Assumes Non-Null

`server/_core/trpc.ts:16`

TrpcContext.user is nullable, but protectedProcedure code accesses ctx.user.id without optional chaining. If authentication middleware fails silently, this causes a null dereference at runtime.

© 2026 NANDI · Internal Use Only

5 PERFORMANCE ISSUES

N+1 Queries · Polling · Bundle Size · DB Indexes

N+1 Query: createNotification Called in Loop for Each Passenger

`server/routers.ts:1544-1566`

When a passenger joins, code loops over existing passengers and calls createNotification() for each — one DB roundtrip per passenger. With 10 passengers and dozens of joins per minute, this creates thousands of sequential DB calls.

Fix: Batch: collect all notifications into array, then INSERT in single bulk query. Most ORMs support bulk insert.

No Spatial Indexing for Driver Location Queries

`server/services/driverMatcher.ts:239-246`

getDriversNear() iterates ALL in-memory driver locations and calculates haversine distance for each, O(n) per request. With 10,000 drivers, this is 10,000 distance calculations per ride booking.

Fix: Implement geohash-based bucketing or k-d tree. Pre-compute bounding box (lat ± radius/111, lng ± radius/(111*cos(lat))) to filter candidates before distance calc.

Active Ride Polling Every 8s — Should Use Existing SSE

`app/active-ride.tsx:162-168`

Client polls getRide and getDriverLocation every 8s via tRPC REST. The server already has an SSE endpoint (GET /api/events) and WebSocket for push updates — polling is redundant and wastes bandwidth.

Fix: Subscribe to SSE ride events in active-ride.tsx using useRideEvents hook (already exists). Only fall back to polling when SSE connection drops.

Missing Composite Database Indexes on Ride Searches

`drizzle/schema.ts`

Ride search queries filter on (status, mode, safetyMode) but no composite index covers this combination. Each search does a full table scan on rides, which degrades linearly with ride count.

Fix: Add: index('rides_status_mode_safety_idx').on(rides.status, rides.mode, rides.safetyMode). Also add: index('rides_driver_step_idx').on(rides.driverId, rides.rideStep).

ETA Recalculation on Every Location Update Without Throttle

`server/routers.ts:3000`

recalculateETAs() may be called on every GPS update from driver (every 5s), fetching OSRM route data each time even if driver position barely changed.

Fix: Throttle: only recalculate if driver moved > 10m since last ETA calc OR > 30s elapsed. Already partially throttled — verify threshold is applied.

No Pagination on My Rides History Query

`server/routers.ts:1085-1090`

getMyRides returns all rides without pagination. A user with 500+ rides loads entire history into memory on every app open. Each ride includes nested passenger and fare data.

Fix: Add cursor-based pagination: input.cursor? (createdAt DESC). Default limit 20, max 100. Return nextCursor for client-side infinite scroll.

© 2026 NANDI · Internal Use Only

In-Memory Rate Limit Map Unbounded Growth

`server/_core/index.ts:106-163`

rateLimitFallback Map grows without bounds. Cleanup runs every 2 minutes but a DDoS spike before cleanup could exhaust Node.js heap memory.

Fix: Cap at 100,000 entries using LRU eviction. Or switch to Redis-only rate limiting with no in-memory fallback in production.

6 INCOMPLETE IMPLEMENTATIONS

TODOs · Mock Data · Unfinished Flows

TODO: sessionVersion Not Included in JWT Payload

`server/routers.ts:513`

A TODO comment confirms sessionVersion is deliberately excluded from JWT. Password changes and explicit logouts do not invalidate existing tokens. This is a known gap awaiting implementation.

Route Deviation Detection Logs Warning But Takes No Action

`server/routers.ts:2942-2964`

checkRouteDeviation() is called and deviation is detected, but no enforcement happens — no auto-cancel, no passenger alert, no driver warning beyond SSE. The safety feature is effectively disabled.

Push Token Storage Has No Expiry or Validity Check

`server/services/pushService.ts`

Expo push tokens stored in userPreferences.pushToken but never validated for expiry. Dead tokens from uninstalled apps are retried indefinitely, wasting push quota.

Geofence Auto-Arrival Callback Wired But Not Fully Integrated

`server/_core/index.ts:474-490`

onGeofenceArrival callback is configured but geofence monitoring is incomplete. Driver must manually press 'Arrived' button — auto-detection via GPS is not actually triggered.

Mid-Ride Join Fare Recalculation Not Wrapped in DB Transaction

`server/routers.ts:2017-2180`

When a mid-ride join is approved, fare recalculation across all passengers runs as multiple separate DB writes. A crash mid-recalculation leaves passengers with inconsistent fares.

Recurring Rides Missing End Date / Max Occurrence Logic

`app/recurring-rides.tsx`

Recurring ride scheduler creates rides indefinitely. No end date, occurrence count limit, or pause/resume mechanism. A cancelled card will keep triggering ride creation failures indefinitely.

© 2026 NANDI · Internal Use Only

7 INFRASTRUCTURE & CONFIG

Redis · Env Vars · Google Maps · Push Notifications

In-Memory Redis Fallback Breaks Distributed State in Production

`server/redis.ts:33-36`

If Redis connection fails, server silently falls back to in-memory Map. This breaks rate limiting consistency, OTP storage, and session state across multiple server instances. Two servers would have divergent state.

Fix: In production (NODE_ENV=production), throw and refuse to start if Redis is unavailable. Alert ops team immediately. Keep in-memory fallback for dev only.

Database Connection Pool Has No Retry or Circuit Breaker

`server/_core/env.ts:57-58`

dbPoolSize defaults to 20, idleTimeout to 60s. No connection retry logic or circuit breaker. A momentary DB outage kills all in-flight requests without graceful degradation.

Fix: Add mysql2 connectionLimit + queueLimit. Implement health check on /healthz that returns 503 if pool is exhausted. Add retry with exponential backoff for transient connection errors.

OTP Fallback Map Cleanup Interval Too Long (5 Minutes)

`server/routers.ts:162-163`

OTP_FALLBACK clears expired entries every 5 minutes. During that window, expired OTPs can still be used for login. Effective OTP lifetime doubles from configured TTL.

Fix: Reduce cleanup interval to 30 seconds. Or use node-ttl-cache package which expires entries lazily on access.

No Graceful Shutdown for Active Rides on Server Restart

`server/_core/index.ts:654-707`

During graceful shutdown, SSE connections close but rideMonitor keeps tracking rides in memory. On restart, in-progress rides lose their monitoring state and passengers may be orphaned.

Fix: On SIGTERM: (1) rideMonitor.stop(), (2) mark all non-terminal rides with a 'system_restart' note in DB, (3) notify affected passengers via push, (4) then close server.

No Circuit Breaker for OSRM / Google Maps External APIs

`server/routers.ts:1347-1358`

Calls to OSRM and Google Maps fail after 5s timeout with no circuit breaker. During OSRM outage, every ride booking waits 5s before falling back to haversine — creating latency spikes.

Fix: Implement circuit breaker: after 5 consecutive OSRM failures, skip to haversine fallback for 60 seconds. Log circuit state changes for monitoring.

Webhook Processing Not Truly Idempotent (Stored Before Processed)

`server/_core/index.ts:166-200`

Webhook is stored with idempotency key before processing. If process crashes after storage but before handler completes, webhook is permanently marked as processed but never actually handled. Payment could be credited to driver without customer charge.

Fix: Store status as 'pending', process handler, then update to 'processed' only on success. Use outbox pattern or a job queue (BullMQ) for reliable processing.

© 2026 NANDI · Internal Use Only

No Alerting When Dispatch Queue Fails to Find Drivers

`server/services/driverMatcher.ts:120-126`

When all driver candidates decline or timeout, 'dispatch:failed' event is emitted but only logged. There's no escalation path — passenger just keeps waiting with no automatic ETA update or fallback.

Fix: On dispatch:failed: (1) send passenger a push notification that search is continuing, (2) widen search radius and retry, (3) after 3 failures, offer ride cancellation with no charge.

Missing Environment Variable Validation for Google Maps Key

`server/services/googleMaps.ts`

GOOGLE_MAPS_API_KEY absence is logged as a warning but execution continues using Nominatim. In production, rate limits on Nominatim (1 req/s) will cause geocoding failures under load.

Fix: Add GOOGLE_MAPS_API_KEY to required env vars for production. Fail fast if missing. Document Nominatim limits clearly in README.

© 2026 NANDI · Internal Use Only

8 ADVANCED FEATURE ROADMAP

Prioritised suggestions for v2+

Replace REST Polling with SSE/WebSocket Subscriptions in Active Ride Screen

HIGH | EFFORT: SMALL

Client polls getRide and getDriverLocation every 8s via HTTP. The SSE endpoint already exists and WebSocket is already broadcasting driver GPS — wire active-ride.tsx to use both. Reduces server load by ~60% for active riders and makes updates feel instant.

Stripe Payment Intent Pre-Authorization at Booking Time

HIGH | EFFORT: MEDIUM

Currently fare is charged at ride completion. Implement pre-authorization (hold) at booking time and capture at completion. Eliminates post-ride payment failures and reduces driver risk. Requires storing paymentIntentId in ride record.

Distributed Tracing with OpenTelemetry

HIGH | EFFORT: MEDIUM

Add OpenTelemetry instrumentation to all critical paths: booking → dispatch → GPS → payment → rating. Enables latency profiling, bottleneck detection, and cross-service debugging. Sentry already installed — add Sentry Performance tracing with OTEL bridge.

Geospatial Driver Search with PostGIS or Redis GEO

HIGH | EFFORT: MEDIUM

Current in-memory haversine scan is O(n) over all online drivers. Replace with Redis GEO commands (GEORADIUS) or store driver positions in MySQL with spatial index. Reduces dispatch latency from O(n) to O(log n) as driver count grows.

Surge Pricing with Machine Learning Demand Forecast

MEDIUM | EFFORT: LARGE

Current surge multiplier is simple threshold-based. Implement ML-based demand forecasting using historical ride data, time-of-day, weather, events. Expo already has analytics endpoint — aggregate it into a forecasting model. Expected 15-20% revenue increase.

Feature Flags for Gradual Feature Rollout

MEDIUM | EFFORT: SMALL

No feature flags visible in the codebase. Implement LaunchDarkly or a simple DB-backed flag system to enable gradual rollout (e.g., enable mid-ride join for 10% of users first). Reduces risk of new feature deployments.

Ride Cancellation Grace Period and Smart Refund Policy

MEDIUM | EFFORT: MEDIUM

Current cancellation is binary (free or full charge). Implement time-based cancellation policy: free within 2 minutes of booking, partial refund 2-5 minutes, no refund after 5 minutes or when driver is < 0.5km away. Add refund retry queue for Stripe failures.

In-App Chat Between Passenger and Driver (WebSocket)

MEDIUM | EFFORT: MEDIUM

chat-screen.tsx and live-chat.tsx exist but no server-side storage or routing. Implement real-time chat via WebSocket with 24-hour message retention. Add automatic translation for international riders. Reduces call volume and support tickets.

© 2026 NANDI · Internal Use Only

Driver Earnings Predictions and Shift Optimization

MEDIUM | EFFORT: MEDIUM

Build on the hourly earnings heatmap already implemented. Add AI-driven shift suggestions: 'You earn 40% more on Friday 5-8pm in downtown — consider going online then.' Display heatmap overlay on driver map view.

Passenger Safety Score and Trust System

MEDIUM | EFFORT: MEDIUM

Drivers currently have ratings but passengers don't. Implement two-way trust scores: passenger rating visible to driver before acceptance. Add strike system for no-shows, bad behaviour. Reduces driver complaints and improves match quality.

Offline Mode with Request Queuing

LOW | EFFORT: SMALL

If network drops mid-booking or mid-ride, app shows errors. Implement offline queue: cache booking request and auto-submit on reconnect. Show 'Offline — retrying' banner. Use NetInfo hook (use-network-status.ts already exists).

Carbon Footprint Tracker for Shared Rides

LOW | EFFORT: SMALL

Show passengers CO2 saved by choosing shared over solo ride. Calculate based on average car emissions per km, number of passengers sharing. Display cumulative impact on profile. Differentiates NANDI from competitors and aligns with ESG goals.

Driver Background Check and Document Verification API

LOW | EFFORT: LARGE

No automated driver verification visible. Integrate with Checkr or similar: license plate → DMV check, ID → identity verification, background check result stored in driverProfiles. Required for insurance and regulatory compliance in most markets.

© 2026 NANDI · Internal Use Only

© 2026 NANDI · Internal Use Only

9 APPENDIX
Tech stack & file manifest

Layer	Technology	Version
Mobile/Web UI	React Native + Expo	SDK 54
Navigation	Expo Router (file-based)	~6.0
State	React Context + AsyncStorage	—
API	tRPC + Zod	11.7
Server	Node.js + Express	20.x
Database	MySQL + Drizzle ORM	^0.44
Auth	JWT + Secure Cookie	—
Payments	Stripe Connect	latest
Push	Expo Push (expo-server-sdk)	^6.1
Maps	OSRM / Google Maps fallback	—
Realtime	SSE + WebSocket (ws)	—
Cache	Redis (in-memory fallback)	—
Monitoring	Sentry	^8.8

Key Server Files

```
server/_core/index.ts — Express app, tRPC mount, rate-limiter, SSE
server/routers.ts — All tRPC procedures (rides, driver, stripe, etc.)
server/db.ts — Drizzle queries + atomicTransitionRideStep
server/services/driverMatcher.ts — Dispatch queue, candidate scoring
server/services/fareEngine.ts — Fare calculation + surge
server/services/rideStateMachine.ts — State graph + atomic transitions
server/services/stripe.ts — Stripe Connect charges
server/services/websocket.ts — WS server for driver GPS broadcast
drizzle/schema.ts — MySQL table definitions + indexes
```

Key Client Files

```
app/(tabs)/index.tsx — Home screen
app/book-ride.tsx — Location + mode selection
app/confirm-ride.tsx — Booking confirmation + payment
app/active-ride.tsx — Live ride tracking (passenger)
app/driver-mode.tsx — Driver dashboard + lifecycle
app/rate-ride.tsx — Post-ride rating screen
app/emergency-sos.tsx — SOS + emergency contacts
lib/rides-store.tsx — Global ride state (Context + AsyncStorage)
lib/trpc.ts — tRPC client setup
```

© 2026 NANDI · Internal Use Only